LISRec: Modeling User Preferences with Learned Item Shortcuts for Sequential Recommendation

Haidong Xin
Northeastern University
Shenyang, China
xinhaidong@stumail.neu.edu.cn

Zhenghao Liu*
Northeastern University
Shenyang, China
liuzhenghao@mail.neu.edu.cn

Sen Mei
Tsinghua University
Beijing, China
meisen2025@gmail.com

Yukun Yan
Tsinghua University
Beijing, China
yanyk.thu@gmail.com

Shi Yu
Tsinghua University
Beijing, China
yus21@mails.tsinghua.edu.cn

Shuo Wang
Tsinghua University
Beijing, China
wangshuo.thu@gmail.com

Zulong Chen
Alibaba Group
Hangzhou, China
zulong.czl@alibaba-inc.com

Yu Gu
Northeastern University
Shenyang, China
guyu@mail.neu.edu.cn

Ge Yu
Northeastern University
Shenyang, China
yuge@mail.neu.edu.cn

Chenyan Xiong
Carnegie Mellon University
Pittsburgh, United States
cx@cmu.edu

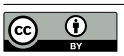
Abstract

User-item interaction histories are pivotal for sequential recommendation systems but often include noise, such as unintended clicks or actions that fail to reflect genuine user preferences. To address this, we propose Learned Item Shortcuts for Sequential Recommendation (LISRec), a novel framework that explicitly captures stable preferences by extracting personalized semantic shortcuts from historical interactions. LISRec first learns task-agnostic semantic representations to assess item similarities, then constructs a personalized semantic graph over all user-interacted items. By identifying the maximal semantic connectivity subset within this graph, LISRec selects the most representative items as semantic shortcuts to guide user preference modeling. This focused representation filters out irrelevant actions while preserving the diversity of genuine interests. Experimental results on the Yelp and Amazon Product datasets illustrate that LISRec achieves a 13% improvement over baseline recommendation models, showing its effectiveness in capturing stable user interests. Further analysis indicates that shortcut-based histories better capture user preferences, making more accurate and relevant recommendations. All codes and datasets are available at https://github.com/NEUIR/LISRec.

CCS Concepts

• **Information systems** → **Recommender systems**.

* indicates corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2258-5/2026/08
https://doi.org/10.1145/3770854.3780337

Keywords

Sequential Recommendation, Task-agnostic Item Representation, User Preference Modeling, Text-based Recommendation

ACM Reference Format:
Haidong Xin, Zhenghao Liu, Sen Mei, Yukun Yan, Shi Yu, Shuo Wang, Zulong Chen, Yu Gu, Ge Yu, and Chenyan Xiong. 2026. LISRec: Modeling User Preferences with Learned Item Shortcuts for Sequential Recommendation. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770854.3780337

1 Introduction

Sequential recommendation systems play a pivotal role in modern digital platforms by dynamically aligning recommendations with evolving user preferences, thereby satisfying user needs and mitigating information overload [10, 35]. While existing approaches predominantly focus on modeling user behavior through interaction histories, a critical challenge remains unaddressed: real-world interaction sequences inherently contain noise from incidental clicks that diverge from user stable preferences. As shown in Figure 1, a user consistently purchasing LEGO products might occasionally click on unrelated items like gaming consoles due to momentary curiosity. Such noisy interactions dilute the signal of genuine preferences, leading to suboptimal recommendation results.

Current methodologies primarily follow two paradigms: item-ID based modeling [16, 31] and item-content based modeling [14, 23]. While these approaches learn representations from interaction sequences, they indiscriminately incorporate all items during training, implicitly assuming uniform relevance across interactions. This oversight amplifies noise propagation, as models struggle to distinguish between core preferences and transient outliers [20, 51].

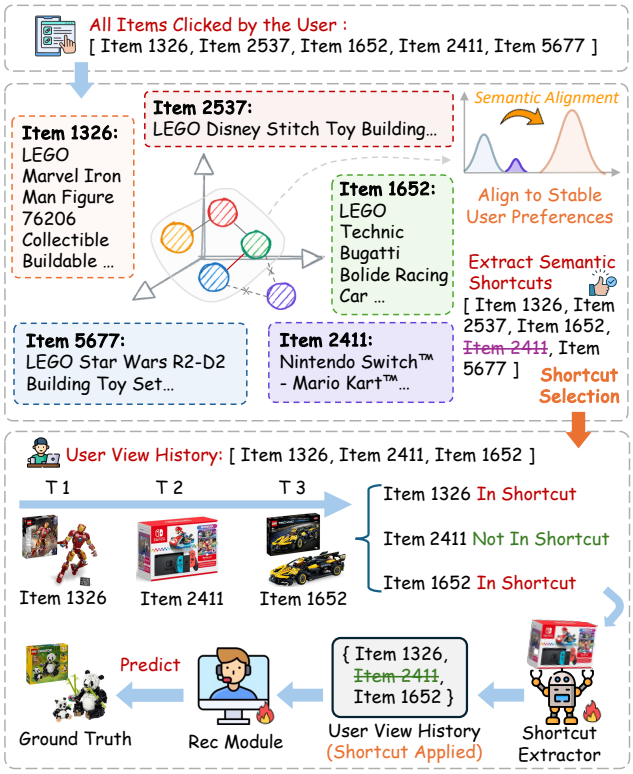


Figure 1: Illustration of Our LISRec Method. LISRec constructs a personalized semantic interaction graph and extracts shortcuts, representing dominant item subsets that provide denoised user representations for preference modeling.

Recent attempts to address noise either adjust item weights [36], or rely on implicit contrastive learning [43], but these methods lack explicit mechanisms to identify and eliminate noise, limiting their interpretability.

To address these challenges, we propose Learned Item Shortcuts for Sequential Recommendation (LISRec), a novel framework that explicitly captures stable user preferences by extracting personalized semantic shortcuts from interaction histories. Specifically, LISRec first learns task-agnostic semantic representations to estimate similarities between items, and then constructs a personalized semantic graph using all items previously clicked by the user, based on pairwise item similarities. Next, LISRec identifies a Maximal Semantic Connectivity subset of the personalized semantic graph that best captures the user's underlying preferences, serving as semantic shortcuts to guide the user behavior modeling for recommendation. By focusing on these highly representative items, LISRec effectively filters out irrelevant interactions while preserving the diversity of the user's genuine interests.

Our experimental results on the Amazon Product and Yelp datasets demonstrate that LISRec achieves up to 13% improvement over a range of baseline models. Our framework generalizes well across different recommendation architectures, highlighting its flexibility. Further analysis shows that the extracted shortcuts consist of semantically similar items, and that these subsets are more predictive of user preferences in next-item recommendation. Overall, LISRec bridges the gap between semantic representation and noise filtering, providing a principled and generalizable solution for improving sequential recommendation performance.

2 Related Work

Sequential recommendation systems aim to model user behaviors by leveraging user-item interaction histories. Early studies rely on the Markov Chain assumption [11] and Matrix Factorization techniques [29] for item prediction. To better represent user preferences, recent works have primarily focused on ID-based modeling, which encodes the randomly initialized ID embeddings of user-interacted items using various neural architectures. These architectures include Recurrent Neural Networks (RNNs) [8, 13, 22, 41], Convolutional Neural Networks (CNNs) [33, 40, 45], Graph Neural Networks (GNNs) [1, 2, 54], and self-attention mechanisms [9, 16, 31, 42].

Instead of representing items using IDs, existing methods focus more on item content-based recommendation systems, leveraging Pretrained Language Models (PLMs) or Large Language Models (LLMs) to enhance item representations by learning textual semantics from their side information [4, 7, 23, 48]. These approaches [4, 23] encode the textual representations of items and further train language models using user-item interaction signals. TASTE [23] verbalizes both items and user interaction histories, utilizing the T5 model [28] to align user and item representations. Leveraging the emergent abilities of LLMs [15, 19], HLLM [4] introduces a hierarchical architecture for sequential recommendation comprising the item LLM and the user LLM.

User interaction sequences in recommendation systems often contain noise, such as accidental clicks or short-term behaviors that do not reflect users' true preferences [20]. To address this, several methods reduce the influence of noisy interactions by monitoring training dynamics (e.g., high prediction loss or unstable gradients), including adaptive sample reweighting [36], label correction via double correction [12], and user-specific denoising strategies [52]. Beyond loss-based methods, self-supervised denoising approaches [20, 51] learn item dependencies to identify noisy signals, while other works adjust item weights based on estimated importance [3, 43, 59]. However, these approaches typically treat denoising as an implicit process without directly removing noisy interactions.

Another line of research [27, 32, 49, 50] focuses on explicitly filtering out noise by identifying and removing low-quality items from interaction sequences. These methods predominantly rely on item ID-based representations, which require extensive user-item interaction histories to be effective [18, 26, 30, 56]. Consequently, they often overlook the potential of text-based semantic information for identifying preference-consistent behaviors [23, 48].

3 Methodology

This section introduces LISRec, a framework for robustly modeling user preferences by extracting semantic shortcuts in a latent space. As shown in Figure 2, we first describe the overall architecture of LISRec, which identifies preference-consistent shortcuts to construct denoised user representations (Sec. 3.1). We then detail the

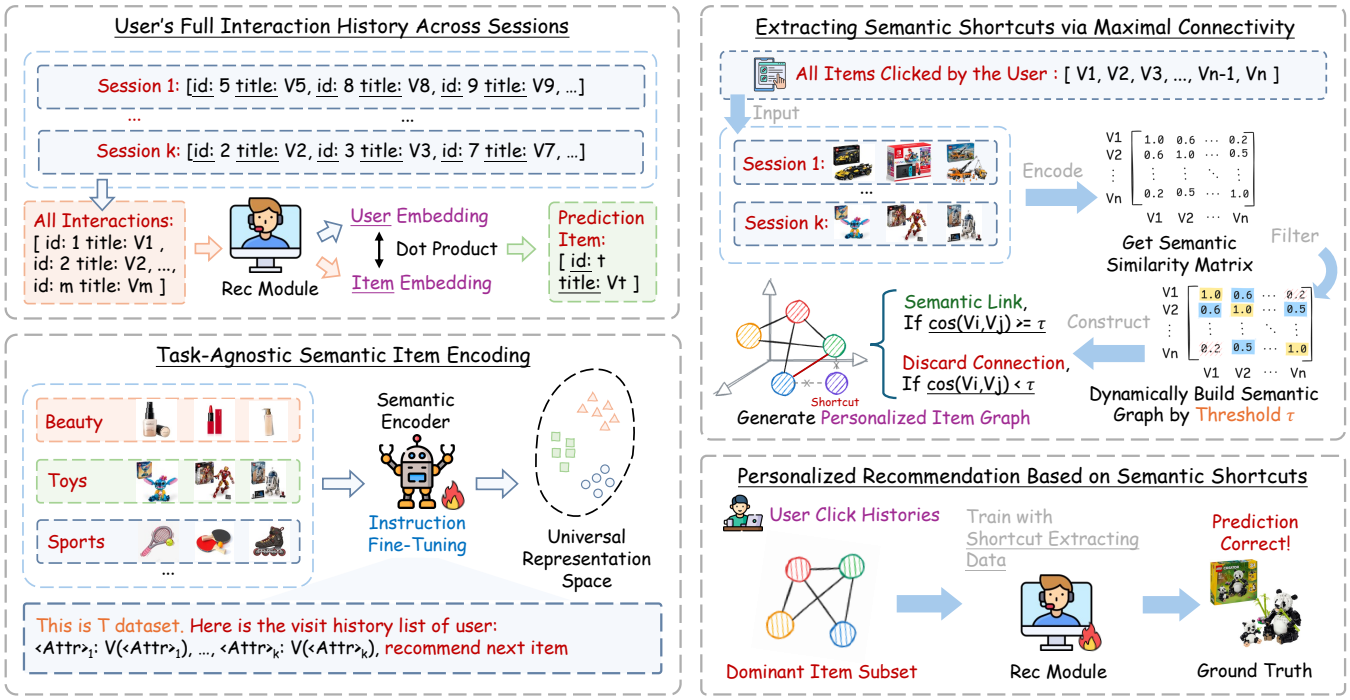


Figure 2: The Model Architecture of LISRec.

shortcut extraction process, which identifies the Maximal Semantic Connectivity within each user's personalized item graph (Sec. 3.2).

3.1 The Framework of LISRec

The sequential recommendation task aims to predict the item v_t that satisfies the preference of the user $\mathcal{U}$ at the t-th time step according to the user-item interaction history $\mathcal{H}$:

$$\mathcal{H} \rightsquigarrow M_{\text{Rec}} \rightsquigarrow v_t, \tag{1}$$

where the user-item interaction history is represented as $\mathcal{H} = \{v_1, v_2, \ldots, v_{t-1}\}$. The recommendation model M_{Rec} estimates the relevance between the user-item interaction history $\mathcal{H}$ and an item v by calculating the ranking probability $P(v|\mathcal{H})$:

$$P(v|\mathcal{H}) = \text{Softmax}_v(\vec{\mathcal{H}} \cdot \vec{v}), \tag{2}$$

where $\cdot$ denotes the dot product operation. Then we follow Liu et al. [23] to get the representations $\vec{v}$ and $\vec{\mathcal{H}}$ for the candidate item v and the user-item interaction history $\mathcal{H}$.

3.1.1 Item Representation. To fully use the semantic information of items, we verbalize the item v using its identifier and k associated attributes <Attr> through the following template:

$$X(v) = \texttt{id}{:}\, v(\texttt{id}) \ldots \texttt{<Attr>}_k : v(\texttt{<Attr>}_k), \tag{3}$$

where $\texttt{<Attr>}_k$ is the name of the k-th attribute, while $v(\texttt{id})$ and $v(\texttt{<Attr>}_i)$ correspond to the text descriptions of the item identifier and the i-th attribute, respectively. Then we encode each item v into an embedding $\vec{v}$ using the T5 model [28]:

$$\vec{v} = \text{T5}(X(v)). \tag{4}$$

3.1.2 Modeling User Preferences via Shortcut Extraction. Users' interaction histories ($\mathcal{H}$) often contain a mixture of long-term interests, short-term preferences, and incidental click behaviors. To model user intent using truly essential items, LISRec tries to model preference-consistent behaviors by building semantic shortcuts in the user-interacted item trajectory to omit some noisy items.

To identify these shortcuts, we introduce a semantic shortcut extractor M_{Filter}, which selectively retrieves a subset of dominant interactions from the full history $\mathcal{H}$. These shortcuts, denoted as $\mathcal{H}_{\text{Shortcut}}$, form a condensed yet highly informative sketch of user preferences in the latent semantic space. This distilled representation is then passed to the recommendation module M_{Rec} for more effective user behavior modeling. The user representation can be formalized as:

$$\mathcal{H} \xrightarrow{M_{\text{Filter}}} \mathcal{H}_{\text{Shortcut}}, \tag{5}$$

where $\mathcal{H}_{\text{Shortcut}} \subseteq \mathcal{H}$ denotes the extracted shortcut subset, containing interactions that are semantically aligned with the user's underlying intent. The semantic shortcut extractor M_{Filter} are detailed in Section 3.2.

To construct the user representation, we first verbalize the extracted shortcut set $\mathcal{H}_{\text{Shortcut}}$ by concatenating the textual representations (Eq. 3) of the selected items:

$$X(\mathcal{H}_{\text{Shortcut}}) = \mathop{\Big\|}_{v_i \in \mathcal{H}_{\text{Shortcut}}} X(v_i), \tag{6}$$

where $\|$ denotes the concatenation operation over the subset of items in $\mathcal{H}_{\text{Shortcut}}$, which may be non-contiguous and unordered with respect to the original history. The concatenated text is then encoded using the T5 model to obtain a semantic representation of

the user's preference:

$$\vec{\mathcal{H}}_{\text{Shortcut}} = \text{T5}(X(\mathcal{H}_{\text{Shortcut}})). \tag{7}$$

3.2 Extracting Preference Shortcuts via Maximal Semantic Connectivity

To robustly model stable user preferences from noisy interaction histories $\mathcal{H}$, we introduce a semantic shortcut extractor M_{Filter}, which identifies the most representative subset of interactions for preference modeling. Specifically, LISRec first encodes items into a task-agnostic semantic space to estimate pairwise item similarity, then constructs a personalized semantic graph for each user and extracts the maximally connected component as the preference-consistent shortcut. This process effectively filters out noisy or inconsistent behaviors during user representation construction.

3.2.1 Task-Agnostic Semantic Item Encoding. To obtain semantically meaningful item representations suitable for preference-level reasoning, we train a language model on item textual descriptions using the instruction tuning based item representation method.

For item semantic representation, each item v is represented purely through its descriptive attributes—excluding any item ID—to improve generalization and reduce dependency on sparse or domain-specific identifiers:

$$\tilde{X}(v)' = \texttt{<Attr>}_1 : v(\texttt{<Attr>}_1) \ \ldots \ \texttt{<Attr>}_k : v(\texttt{<Attr>}_k). \tag{8}$$

Building upon the Next Item Prediction (NIP) objective [31], we serialize each user history $\mathcal{H} = \{v_1, \ldots, v_{t-1}\}$ into a concatenated text sequence:

$$\tilde{X}'(\mathcal{H}) = \tilde{X}(v_{t-1})'; \ldots; \tilde{X}(v_1)'. \tag{9}$$

This sequence is passed to the encoder alongside an instruction prompt: "This is the $\mathcal{T}$ dataset. Here is the visit history list of the user: $\tilde{X}'(\mathcal{H})$, recommend next item". The encoder is trained to predict the next item v_t, encouraging it to learn semantic dependencies that align with user preferences. After training, the encoder produces item embeddings $\vec{v}$, where pairwise semantic similarity can be evaluated via the cosine similarity:

$$S_{ij} = \cos(\vec{v}_i, \vec{v}_j). \tag{10}$$

This representation approach is fully text-driven, enabling the item representation model to operate across item vocabularies and different tasks while capturing semantic regularities grounded in language.

3.2.2 Modeling Stable Preferences with Personal Item Graphs. To extract the user's core preference signals from noisy interaction histories, we construct a personalized semantic interaction graph for each user $\mathcal{U}$, where edges capture semantic coherence between interacted items:

$$G_{\mathcal{U}} = (\mathcal{V}, \mathcal{E}), \tag{11}$$

where $\mathcal{V} = \{v_1, \ldots, v_n\}$ includes all items interacted by user $\mathcal{U}$ in all previous sessions, and $\mathcal{E}$ denotes the edge set encoding semantic connections. An undirected edge $e_{ij} \in \mathcal{E}$ is created between items v_i and v_j if their embedding similarity exceeds a threshold τ:

$$e_{ij} = \begin{cases} 0 & \text{if } S_{ij} < \tau, \\ 1 & \text{if } S_{ij} \geq \tau, \end{cases} \tag{12}$$

where $e_{ij} = 0$ is the cosine similarity between the embeddings of v_i and v_j are not connected (Eq. 10). This design naturally filters out semantically dissimilar and outlier items with respect to user preferences.

To identify the user's dominant preference signal, we extract the largest connected component from the graph $G_{\mathcal{U}}$:

$$C_{\max} = \arg\max_{C_i} |\mathcal{V}(C_i)|, \tag{13}$$

and define the corresponding item subset as the preference shortcut:

$$\mathcal{H}_{\text{Shortcut}} = \{v_j \mid v_j \in \mathcal{H} \wedge v_j \in \mathcal{V}(C_{\max})\}. \tag{14}$$

This shortcut subset serves as a semantically consistent and compact proxy for the user's intent. By selectively modeling only the maximally connected region in the semantic graph, we avoid overfitting to noisy interactions while mitigating over, ensuring richer and more robust preference representations for downstream recommendation.

4 Experimental Methodology

This section describes the datasets, evaluation metrics, baseline models, and implementation details used in our experiments.

Datasets. We utilize four sequential recommendation datasets–Beauty, Sports, and Toys from Amazon Product [25], and Yelp–to train and evaluate both the sequential recommendation module and the semantic shortcut extractor. More details are shown in Appendix A.2.

Evaluation Metrics. We adopt Recall@10/20 and NDCG@10/20 to evaluate the recommendation performance, which is consistent with prior works [4, 23, 39]. Statistical significance is determined using a permutation test with $P < 0.05$.

Baselines. We compare LISRec with several widely used methods for sequential recommendation, including vanilla sequential recommendation models and some item denoising techniques.

Vanilla Sequential Recommendation Models. We compare LISRec with both ID-based and content-based recommendation models. First, LISRec is evaluated against seven ID-based recommendation models. GRU4Rec [13] employs Recurrent Neural Networks (RNNs) to model user-item interaction sequences, while SASRec [16] and Bert4Rec [31] leverage self-attention to capture user preferences from sequential interactions. Additionally, we examine models that incorporate item-side information. S^3Rec [58] enhances self-attention modules through four self-supervised pretraining strategies to better model relationships among attributes, items, and users. ICAI-SR [47] represents item-attribute relationships using a heterogeneous graph to improve relevance modeling, whereas NOVA [21] integrates item attributes as side information via attention modules. DIF-SR [39] introduces a non-invasive attention mechanism to fuse item attribute information into user behavior modeling effectively.

Furthermore, LISRec is compared with four content-based recommendation models. CoWPiRec [42] optimizes Pretrained Language Models (PLMs) to enhance item representation by aligning it with collaborative filtering representations derived from a word graph. TASTE [23] leverages T5 [28] to directly encode the text representations of both users and items for matching items with user intentions. DWSRec [53] leverages both fully whitened and relaxed whitened item representations as dual views for effective recommendations. Lastly, HLLM [4] employs Large Language Models

Table 1: Overall Performance of LISRec. The best results are marked in bold, while the second-best results are underlined. The HLLM* model is reproduced using a 1B-scale LLM. † and ‡ denote statistically significant improvements over DIF-SR† and TASTE‡, respectively.

Dataset	Metrics	Item ID based Recommendation Models							Text based Recommendation Models				
		GRU4Rec	Bert4Rec	SASRec	S^3Rec	NOVA	ICAI-SR	DIF-SR	CoWPiRec	DWSRec	HLLM*	TASTE	LISRec
Beauty	R@10	0.0530	0.0529	0.0828	0.0868	0.0887	0.0879	0.0908	0.0837	0.0989	0.0887	0.1030	**0.1249**†‡
	R@20	0.0839	0.0815	0.1197	0.1236	0.1237	0.1231	0.1284	0.1280	0.1496	0.1238	0.1550	**0.1774**†‡
	N@10	0.0266	0.0237	0.0371	0.0439	0.0439	0.0439	0.0446	0.0405	0.0470	0.0499	0.0517	**0.0641**†‡
	N@20	0.0344	0.0309	0.0464	0.0531	0.0527	0.0528	0.0541	0.0517	0.0597	0.0588	0.0649	**0.0772**†‡
Sports	R@10	0.0312	0.0295	0.0526	0.0517	0.0534	0.0527	0.0556	0.0588	0.0629	0.0555	0.0633	**0.0721**†‡
	R@20	0.0482	0.0465	0.0773	0.0758	0.0759	0.0762	0.0800	0.0892	0.0958	0.0816	0.0964	**0.1082**†‡
	N@10	0.0157	0.0130	0.0233	0.0249	0.0250	0.0243	0.0264	0.0300	0.0313	0.0295	0.0338	**0.0372**†‡
	N@20	0.0200	0.0173	0.0295	0.0310	0.0307	0.0302	0.0325	0.0376	0.0396	0.0360	0.0421	**0.0463**†‡
Toys	R@10	0.0370	0.0533	0.0831	0.0967	0.0978	0.0972	0.1013	0.0502	0.0967	0.0928	0.1232	**0.1304**†‡
	R@20	0.0588	0.0787	0.1168	0.1349	0.1322	0.1303	0.1382	0.0739	0.1472	0.1306	0.1789	**0.1896**†‡
	N@10	0.0184	0.0234	0.0375	0.0475	0.0480	0.0478	0.0504	0.0272	0.0457	0.0520	**0.0640**	0.0637†
	N@20	0.0239	0.0297	0.0460	0.0571	0.0567	0.0561	0.0597	0.0331	0.0585	0.0616	0.0780	**0.0786**†
Yelp	R@10	0.0361	0.0524	0.0650	0.0589	0.0681	0.0663	0.0698	0.0657	0.0702	0.0659	0.0738	**0.0749**†‡
	R@20	0.0592	0.0756	0.0928	0.0902	0.0964	0.0940	0.1003	0.0959	0.1103	0.0955	0.1156	**0.1173**†‡
	N@10	0.0184	0.0327	0.0401	0.0338	0.0412	0.0400	**0.0419**	0.0416	0.0382	0.0407	0.0397	0.0408‡
	N@20	0.0243	0.0385	0.0471	0.0416	0.0483	0.0470	0.0496	0.0497	0.0482	0.0504	0.0502	**0.0509**†‡

(LLMs) to generate item representations, subsequently utilizing a hierarchical architecture to predict the next item.

Item Denoising for Sequential Recommendation. We also evaluate LISRec against three baselines that either explicitly remove noisy items or implicitly address item noise through denoising modeling. All these models are implemented based on the item-ID base recommendation model, such as Bert4Rec [31]. For methods that explicitly remove noisy items, DSAN [46] learns a target item embedding and applies an adaptively sparse transformation function to filter out noisy interactions. Similarly, HSD [50] identifies and excludes noisy interactions by detecting inconsistencies with user interests, leveraging the sequential order of recorded interactions. In contrast, DCRec [43] employs a debiased contrastive learning framework to implicitly mitigate the impact of noise.

Implementation Details. LISRec is implemented using OpenMatch[1] [24, 44] and optimized using in-batch negatives. Both the item representation model in the semantic shortcut extractor and the recommendation module are initialized from the T5-base checkpoint[2] in Huggingface Transformers [37]. The item representation model in the semantic shortcut extractor is trained using the Adam optimizer with a learning rate of 1e-4 and a batch size of 32. For the recommendation module, we follow the settings in Liu et al. [23], which is trained using the Adam optimizer with a learning rate of 1e-4, a warm-up proportion of 0.1, and a batch size of 8. We set the filtering threshold τ to 0.7, based on the hyperparameter sensitivity analysis reported in Appendix 5.3. During inference, we excluded the last interaction from the user-interaction sequence when constructing the consistency graph, thereby avoiding potential data leakage.

Additionally, we reproduce all item-ID based models and DWSRec [53] using implementations provided by RecBole[3] [57], maintaining consistent settings across all models for fair comparison. For CoWPiRec and TASTE, we adhere to the default experimental settings as specified in their original papers. For HLLM, due to computational constraints, we ultilize TinyLlama-1.1B checkpoint[4] [55] as the item-LLM component, and the user-LLM is implemented as a custom Llama model [34] with two hidden layers, trained from scratch using the following settings: learning rate of 1e-4, batch size of 2, training for 5 epochs, and a negative sampling ratio of 1024 negative samples per positive sample.

[1]https://github.com/OpenMatch/OpenMatch
[2]https://huggingface.co/google-t5/t5-base
[3]https://recbole.io/

5 Experimental Results

In this section, we begin by evaluating the recommendation performance of LISRec and perform ablation studies to assess the effectiveness of the semantic shortcut extractor and various training strategies. Then, we analyze textual similarities among retained items to evaluate the effectiveness of the item representation learning method. Finally, several case studies are shown.

5.1 Overall Performance

In this section, we first implement LISRec based on TASTE [23] to evaluate its recommendation performance. Subsequently, we implement LISRec with Bert4Rec [31] to evaluate its denoising capability by comparing it against various baselines.

5.1.1 Recommendation Performance. As shown in Table 1, we present the overall performance of LISRec and various recommendation baseline models. Overall, LISRec demonstrates its effectiveness by significantly outperforming all baselines across four datasets, leveraging TASTE [23] as its backbone architecture.

Among all baseline models, text-based recommendation models generally exhibit significantly stronger performance compared to item ID-based models. This superiority derives from the ability of PLMs and LLMs to effectively capture matching signals through text representations of users and items. Based on the TASTE backbone, LISRec integrates a semantic shortcut extractor to remove noisy items, utilizing these retained user-item interactions to train the

[4]https://huggingface.co/TinyLlama/TinyLlama-1.1B-intermediate-step-1431k-3T

Table 2: Denoising Performance of LISRec. To ensure a fair comparison, all denoising methods are implemented using Bert4Rec [31] as the backbone model.

Dataset	Method	R@10	R@20	N@10	N@20
Beauty	DSAN	0.0215	0.0320	0.0103	0.0132
	HSD	0.0423	0.0645	0.0218	0.0262
	DCRec	0.0609	0.0869	0.0317	0.0387
	LISRec	**0.0627**	**0.0904**	**0.0336**	**0.0406**
Sports	DSAN	0.0107	0.0159	0.0051	0.0065
	HSD	0.0214	0.0317	0.0103	0.0133
	DCRec	0.0262	0.0407	0.0139	0.0175
	LISRec	**0.0275**	**0.0428**	**0.0147**	**0.0185**
Toys	DSAN	0.0172	0.0258	0.0086	0.0104
	HSD	0.0344	0.0517	0.0172	0.0209
	DCRec	0.0443	0.0660	0.0243	0.0296
	LISRec	**0.0468**	**0.0691**	**0.0258**	**0.0314**
Yelp	DSAN	0.0153	0.0218	0.0084	0.0116
	HSD	0.0326	0.0483	0.0165	0.0247
	DCRec	0.0387	0.0562	0.0204	0.0262
	LISRec	**0.0398**	**0.0576**	**0.0211**	**0.0274**

recommendation module. The evaluation results reveal a significant improvement in the recommendation performance, highlighting its effectiveness in enabling the recommendation module to better model user behavior. Among various recommendation tasks, LISRec demonstrates much better performance on the Beauty and Sports datasets, while showing relatively weaker improvements on the Yelp dataset. This discrepancy may be attributed to the tendency of users to make more casual clicks driven by instant interest in these shopping scenarios.

5.1.2 Denoising Effectiveness. We compare LISRec with three state-of-the-art denoising methods–DSAN [46], HSD [50], and DCRec [43]–to evaluate its denoising effectiveness. The baselines are categorized into two groups–explicit denoising methods (DSAN and HSD) and implicit denoising methods (DCRec). All methods are implemented using the item ID-based recommendation model, Bert4Rec [31]. The evaluation results are shown in Table 2.

Among the baseline methods, DCRec achieves the best performance by modeling user behaviors based on raw user-item interactions and effectively mitigating noise through dynamically assigning weights to user-interacted items. However, this implicit denoising approach necessitates model-specific training, restricting our experiments to the Bert4Rec model used in constructing the DCRec framework. Benefiting from explicit denoising methods, LISRec can be applied to the item ID-based recommendation model and achieves a 4.7% improvement over all denoising baseline models, highlighting its effectiveness in filtering noisy items. This consistent improvement with the item ID-based model underscores the potential of extending the filtering approach of LISRec to different architectures of recommendation models.

Table 3: Ablation Study of LISRec. † and ‡ denote statistically significant improvements over LISRec (w/o M_{Filter})† and LISRec (Zero-Shot)‡.

Dataset	Method	R@20	N@20
Beauty	LISRec (Zero-Shot)	0.1631	0.0697
	w/o M_{Filter}	0.1550	0.0649
	LISRec	**0.1774**†‡	**0.0772**†‡
	w/o M_{Rec} Training	0.1655†	0.0727†
	w/o M_{Filter} Training	0.1761†‡	0.0762†‡
Sports	LISRec (Zero-Shot)	0.1014	0.0420
	w/o M_{Filter}	0.0964	0.0421
	LISRec	**0.1082**†‡	**0.0496**†‡
	w/o M_{Rec} Training	0.1047†	0.0433†
	w/o M_{Filter} Training	0.1073†‡	0.0457†‡
Toys	LISRec (Zero-Shot)	0.1797	0.0786
	w/o M_{Filter}	0.1789	0.0780
	LISRec	**0.1896**†‡	**0.0826**†‡
	w/o M_{Rec} Training	0.1836†	0.0796†
	w/o M_{Filter} Training	0.1879†‡	0.0802†‡
Yelp	LISRec (Zero-Shot)	0.1158	0.0504
	w/o M_{Filter}	0.1156	0.0502
	LISRec	**0.1173**†	**0.0509**†
	w/o M_{Rec} Training	0.1165	0.0504
	w/o M_{Filter} Training	0.1169†	0.0507†

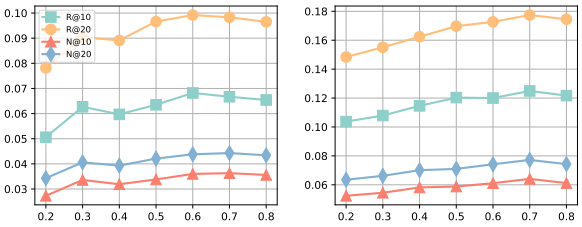
Table 4: Ablation Study on Different Filter Models in LISRec.

Dataset	Method	R@20	N@20
Beauty	LISRec	**0.1774**	**0.0772**
	w/ Vanilla T5 M_{Filter}	0.1761	0.0762
	w/ TASTE M_{Filter}	0.1767	0.0767
Sports	LISRec	**0.1082**	**0.0496**
	w/ Vanilla T5 M_{Filter}	0.1073	0.0457
	w/ TASTE M_{Filter}	0.1077	0.0471
Toys	LISRec	**0.1896**	**0.0826**
	w/ Vanilla T5 M_{Filter}	0.1879	0.0802
	w/ TASTE M_{Filter}	0.1883	0.0811
Yelp	LISRec	**0.1173**	**0.0509**
	w/ Vanilla T5 M_{Filter}	0.1169	0.0507
	w/ TASTE M_{Filter}	0.1170	0.0508

5.2 Ablation Study

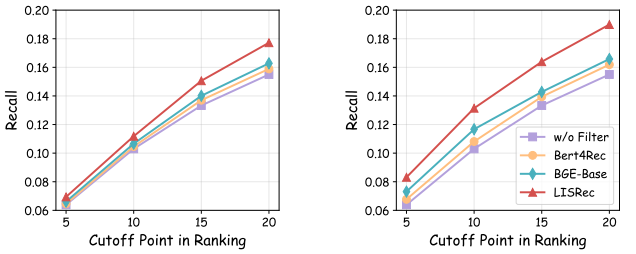
This section presents an ablation study to evaluate the contribution of the semantic shortcut extractor (M_{Filter}) and the effectiveness of our proposed training strategy in LISRec.

Table 3 shows the performance of several ablated versions of our model across four datasets. Comparing LISRec (w/o M_{Filter}) with LISRec (Zero-Shot), we observe a consistent improvement in recommendation performance across all datasets when a zero-shot M_{Filter}



(a) Bert4Rec (b) TASTE

Figure 3: Hyperparameter Sensitivity Analysis. We conduct experiments on the threshold hyperparameter, τ, with results derived from the Amazon Beauty dataset using the Bert4Rec 3(a) and TASTE 3(b) backbone models.



(a) Raw User-Item Interaction Trained M_{Rec}. (b) M_{Filter}-Retained Items Trained M_{Rec}.

Figure 4: Performance of Recommendation Models Using Different Semantic Shortcut Extraction Methods.

is applied during inference. This highlights the inherent benefit of even an untrained semantic shortcut extractor in refining user-interacted items for recommendation. The impact of the semantic shortcut extractor becomes more evident when comparing LISRec (Zero-Shot) with LISRec (w/o M_{Rec} Training). Training the M_{Filter} module with our instruction-based approach results in significant performance gains, demonstrating the value of learning tailored item representations for extracting semantic shortcuts. In particular, LISRec (w/o M_{Rec} Training) consistently outperforms LISRec (w/o M_{Filter}), highlighting that an effective semantic shortcut extractor is essential for improved recommendation accuracy.

Furthermore, the full LISRec model, which trains the recommendation module M_{Rec} on the retained interactions, achieves the best performance across all datasets, with an average improvement of over 12% compared to LISRec (Zero-Shot). This substantial gain underscores the importance of aligning the recommendation module with the retained data, enabling it to better capture user preferences and behaviors from the refined interaction history. The consistent statistically significant improvements of LISRec over both LISRec (w/o M_{Filter}) and LISRec (Zero-Shot) further validate the effectiveness of our complete model.

To evaluate the effectiveness of the M_{Filter} module in LISRec, we compare it with two alternatives: a vanilla T5 M_{Filter} initialized from T5-base without task-specific training, and a TASTE M_{Filter} (trained on recommendation data).

As shown in Table 4, LISRec achieves the best performance across all datasets, consistently surpassing both baselines. The vanilla T5 M_{Filter} lags behind due to its lack of adaptation to recommendation data, while the TASTE-based variant, though improved, is still limited by its exposure to only a single dataset during training. In contrast, our approach benefits from balanced pre-training over multiple datasets, leading to more robust and transferable item representations. These results highlight the importance of both domain-specific and multi-dataset training for semantic shortcut extraction in recommendation.

5.3 Hyperparameter Sensitivity Analysis

We analyze the impact of the similarity threshold τ, which is a key hyperparameter in the construction of the personal item graph described in Sec.3.2.2, by varying τ from 0.2 to 0.8 while keeping all other experimental settings fixed. Figure 3 shows the performance of Bert4Rec and TASTE across four evaluation metrics (Recall@10/20 and NDCG@10/20).

The results reveal a clear and consistent pattern for both backbone models. When τ is set too low (0.2 to 0.4), the semantic shortcut extractor becomes overly permissive, allowing many irrelevant or noisy items into the shortcut, which results in degraded performance. As τ increases to a moderate range (0.5 to 0.6), performance improves markedly, indicating that the shortcut extractor is now able to focus on truly relevant interactions and effectively filter out noise. For example, Bert4Rec achieves its best R@10 around $\tau = 0.6$, and similar patterns are observed for all metrics and for the TASTE backbone.

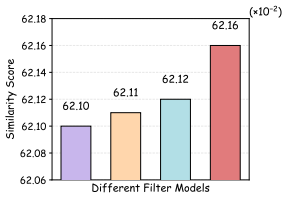
Importantly, our LISRec framework consistently outperforms baseline models without semantic filtering across the entire range of τ values. This demonstrates the robustness and generalizability of our approach–even when the threshold is not optimally tuned, shortcut extraction leads to reliable improvements in recommendation quality.

5.4 Effectiveness of Different Item Representations for Shortcut Extraction

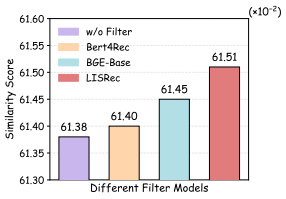
To evaluate the effectiveness of the semantic shortcut extractor in LISRec, we perform a comparative analysis involving various item representation strategies. Specifically, we compare the full LISRec model with its variant without the semantic shortcut extractor (LISRec w/o Filter), as well as versions of LISRec equipped with different item representation methods within the semantic shortcut extractor.

We implement the semantic shortcut extractor M_{Filter} using three approaches: an ID-based recommendation model, Bert4Rec [31], and a dense retriever, BGE-Base [38], which serve as baselines for evaluating representation effectiveness.

As illustrated in Figure 5, we analyze the text similarity among the items retained by M_{Filter} using the BGE-Base model to compute pairwise similarity scores. In Figure 5(a), we report the average similarity among the items selected by different semantic shortcut extraction strategies. The results show that all methods increase the internal similarity among selected items. Notably, when using

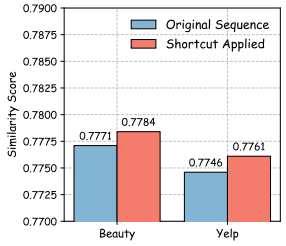


(a) Average Similarity Scores among M_{Filter}-Retained Items.

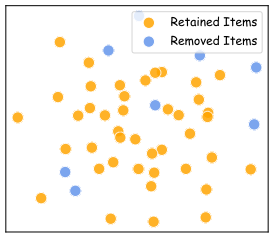


(b) Similarity Scores between M_{Filter}-Retained Items and the Target Items.

Figure 5: Effectiveness of LISRec in Learning Item Representations for Semantic Shortcut Extraction. Figure 5(a) and Figure 5(b) illustrate the item similarities among M_{Filter}-retained items and the ground truth items.



(a) Similarity Scores between User Representation and Target Items.



(b) Embedding Visualization of Items using LISRec.

Figure 6: Effectiveness of Semantic Shortcut Extraction in Enhancing the Recommendation Module. Figure 6(a) and Figure 6(b) show the improvement of user and item representations after extracting shortcuts.

LISRec, the extracted item subset achieves the highest average similarity, indicating stronger semantic alignment within the shortcut.

Furthermore, Figure 5(b) presents the average similarity between the M_{Filter}-retained user-interacted items and the ground-truth target item. Among all methods, LISRec yields the highest similarity, demonstrating its effectiveness in maintaining items that are semantically aligned with the target item that users need.

As shown in Figure 4, we further evaluate the impact of different shortcut extraction methods on overall recommendation performance. Figures 4(a) and 4(b) report the performance of the recommendation module M_{Rec} when trained on either the raw user-item interaction set $\mathcal{H}$ or the shortcut subset $\mathcal{H}_{\text{Shortcut}}$ extracted by M_{Filter}. In both cases, LISRec consistently outperforms other item representation models, with particularly strong gains at larger ranking cutoffs. This demonstrates its effectiveness in enabling the recommendation model to focus on the most relevant items, thereby better capturing stable user preferences. The advantage of LISRec is attributed to its PLM-based ability to capture item text semantics and to identify semantically meaningful patterns in user-item interactions. After training the recommendation module M_{Rec} using the shortcut-extracted interaction histories, LISRec achieves substantial performance improvements, further highlighting the effectiveness of semantic shortcut extraction.

5.5 Effectiveness of Shortcut Extraction in Enhancing the Recommendation Module

As illustrated in Figure 6, we investigate how the shortcut-retained items contribute to improving the recommendation module during training.

For each user, we first compute the embedding similarity between the user representation and the corresponding ground-truth item, under both the original and shortcut-applied training settings. Figure 6(a) shows that applying shortcut extraction during training consistently increases this similarity. This finding confirms that shortcut extraction strengthens the semantic alignment between user and target item representations, leading to more faithful modeling of user intent.

Furthermore, we randomly select a user and then use t-SNE to visualize item embeddings. As shown in Figure 6(b), LISRec is able to better distinguish the removed items by mapping them to the periphery of the embedding space, effectively treating them as outliers. This observation highlights the effectiveness of LISRec in leveraging denoising signals as prior knowledge to guide the recommendation module. By doing so, it facilitates a more discriminative representation space, where retained and removed items are clearly separated through contrastive training.

5.6 Case Studies

In Table 5, we present two examples from the Amazon Beauty and Yelp datasets to evaluate the effectiveness of LISRec. For each dataset, we randomly select a user and showcase how semantic shortcut extraction operates on their interaction history. In the case studies, we detail the procedure of constructing the personalized item interaction graph and extracting the maximal preference-consistent subgraph, which serves to filter out noisy interactions.

As shown in the first case from the Amazon Beauty dataset, the user primarily interacts with face cream products focused on moisturizing, such as *"Bare Escentuals bareMinerals Purely Nourishing Moisturizer"* and *"Ole Henriksen Truth Creme Advanced Hydration"*. However, the user accidentally clicks on a product–*"VALENTINO VALENTINA by Valentino for WOMEN: EAU DE PARFUM SPRAY 1.7 OZ"*–due to their instant interests, which are significantly different in terms of both product types and attributes. In detail, while other items in the sequence emphasize *"hydrating"* and *"moisturizing"*, the *"Parfum Spray"* is unrelated to these preferences. When LISRec constructs the user-interacted item graph, the *"Parfum Spray"* product fails to form edges with other items, indicating that this interaction is likely an outlier and should be excluded from user-interacted items. Consequently, the denoised item sequence $\mathcal{H}_{\text{Shortcut}}$ includes only products aligned with the consistent preferences of a specific user, such as moisturizing creams, enabling the recommendation module to better model user behavior from the user-item interactions.

In the second case, we present a specific user from the Yelp dataset who visits various locations. This user primarily interacts with places characterized by attributes such as *"Nightlife"*, *"Restaurants"*, and *"Bars"*, demonstrating a consistent pattern of preferences. However, one of the interacted locations–the *"Spay & Neuter Center of Southern Nevada"*–is a pet hospital, which is distinct from the user-consistent preference for nightlife and dining. Although this

Table 5: Case Studies. We present cases from two datasets, Amazon Beauty and Yelp. Different colors are highlighted to distinguish preference types: Blue represents majority preference, and Orange represents discrete interaction.

Case #1 in Amazon Beauty
Node_ID: n_1 **Name:** Bare Escentuals bareMinerals Purely Nourishing Moisturizer: Combination Skin
Description: ... Hydrate skin with this lightweight, remarkably effective moisturizer for improved firmness, elasticity ...
Node_ID: n_2 **Name:** Ole Henriksen Truth Creme Advanced Hydration, 1.7 Fluid Ounce
Description: ... An effective and groundbreaking anti-aging moisturizer contains advanced hydration complexity that deliver ...
Node_ID: n_3 **Name:** VALENTINO VALENTINA by Valentino for WOMEN: EAU DE PARFUM SPRAY 1.7 OZ
Description: ... Valentina By Valentino 1.7 Oz Eau De Parfum Spray ...
Node_ID: n_4 **Name:** Boscia Oil-Free Nightly Hydration, 1.4-Fluid Ounce
Description: ... An oil-free nightly moisturizer that provides weightless hydration, combats excess oil and refines the look of ...
Similarity: $S_{1,2} = 0.4441$, $S_{1,3} = 0.2527$, $S_{1,4} = 0.4608$, $S_{2,3} = 0.2125$, $S_{2,4} = 0.4477$, $S_{3,4} = 0.2049$
Add Edge: (n_1, n_2), (n_1, n_4), (n_2, n_4)
The maximum connected subgraph $C_{\max}$ is $\{1, 2, 4\}$, so that the filtered interaction sequence $\mathcal{H}_{\text{Shortcut}}$ is: $\{1, 2, 4\}$
Case #2 in Yelp
Node_ID: n_1 **Address:** Las Vegas, NV **Name:** Maggiano's Little Italy
Description: ... Nightlife, Restaurants, Italian, Bars ...
Node_ID: n_2 **Address:** Las Vegas, NV **Name:** Lazy Dog Restaurant & Bar
Description: ... Nightlife, Food, Beer, Wine & Spirits, Pizza, Burgers ...
Node_ID: n_3 **Address:** Las Vegas, NV **Name:** Spay & Neuter Center of Southern Nevada
Description: ... Veterinarians, Pets, Pet Services ...
Node_ID: n_4 **Address:** Las Vegas, NV **Name:** Weera Thai Restaurant - Sahara
Description: ... Restaurants, Beer, Wine & Spirits, Food, Vegetarian, ..., Seafood, Bars, Nightlife ...
Similarity: $S_{1,2} = 0.5926$, $S_{1,3} = 0.4699$, $S_{1,4} = 0.5121$, $S_{2,3} = 0.4865$, $S_{2,4} = 0.5150$, $S_{3,4} = 0.3932$
Add Edge: (n_1, n_2), (n_1, n_4), (n_2, n_4)
The maximum connected subgraph $C_{\max}$ is $\{1, 2, 4\}$, so that the filtered interaction sequence $\mathcal{H}_{\text{Shortcut}}$ is: $\{1, 2, 4\}$

pet hospital is situated on the same street as other frequently visited locations, it likely represents an occasional visit rather than a reflection of the general behavior of the user. LISRec assigns a lower similarity score between this noisy item and other user-interacted items, effectively excluding the noisy item from the sequence of user-interacted history. This enables the recommendation model to focus on more representative items, such as *"Maggiano's Little Italy"* and *"Weera Thai Restaurant"*.

6 Conclusion

This paper introduces the Learned Item Shortcuts for Recommendation System (LISRec), a new framework that models stable user preferences by identifying dominant item subsets within a latent semantic space. LISRec leverages universal item representations to construct semantically meaningful shortcuts in user interaction histories. Our experiments on Yelp and Amazon Product datasets show that LISRec consistently outperforms baseline and previous text-based models. The results demonstrate that instruction-tuned universal item representations enable more robust shortcut selection and lead to more accurate, preference-consistent recommendations.

Acknowledgments

This work is partly supported by the National Natural Science Foundation of China (No. 62206042 and No. 62461146205). This work is also supported by the AI9Stars community.

References

[1] Xuheng Cai, Chao Huang, Lianghao Xia, and Xubin Ren. 2023. LightGCL: Simple Yet Effective Graph Contrastive Learning for Recommendation. In *Proceedings of ICLR*. https://openreview.net/forum?id=FKXVK9dyMM

[2] Jianxin Chang, Chen Gao, Yu Zheng, Yiqun Hui, Yanan Niu, Yang Song, Depeng Jin, and Yong Li. 2021. Sequential Recommendation with Graph Neural Networks. In *Proceedings of SIGIR*. 378–387. https://doi.org/10.1609/aaai.v33i01.3301346

[3] Huiyuan Chen, Yusan Lin, Menghai Pan, Lan Wang, Chin-Chia Michael Yeh, Xiaoting Li, Yan Zheng, Fei Wang, and Hao Yang. 2022. Denoising self-attentive sequential recommendation. In *Proceedings of RecSys*. 92–101. https://doi.org/10.1145/3523227.3546788

[4] Junyi Chen, Lu Chi, Bingyue Peng, and Zehuan Yuan. 2024. HLLM: Enhancing Sequential Recommendations via Hierarchical Large Language Models for Item and User Modeling. *ArXiv preprint* abs/2409.12740 (2024). https://arxiv.org/abs/2409.12740

[5] Xu Chen, Hongteng Xu, Yongfeng Zhang, Jiaxi Tang, Yixin Cao, Zheng Qin, and Hongyuan Zha. 2018. Sequential Recommendation with User Memory Networks. In *Proceedings of WSDM*. 108–116. https://doi.org/10.1145/3159652.3159668

[6] Alexander Dallmann, Daniel Zoller, and Andreas Hotho. 2021. A Case Study on Sampling Strategies for Evaluating Neural Sequential Item Recommendation Models. In *Proceedings of RecSys*. 505–514. https://doi.org/10.1145/3460231.3475943

[7] Hao Ding, Yifei Ma, Anoop Deoras, Yuyang Wang, and Hao Wang. 2021. Zero-shot recommender systems. *ArXiv preprint* abs/2105.08318 (2021). https://arxiv.org/abs/2105.08318

[8] Tim Donkers, Benedikt Loepp, and Jürgen Ziegler. 2017. Sequential User-based Recurrent Neural Network Recommendations. In *Proceedings of RecSys*. 152–160. https://doi.org/10.1145/3109859.3109877

[9] Xinyu Du, Huanhuan Yuan, Pengpeng Zhao, Jianfeng Qu, Fuzhen Zhuang, Guanfeng Liu, Yanchi Liu, and Victor S. Sheng. 2023. Frequency Enhanced Hybrid Attention Network for Sequential Recommendation. In *Proceedings of SIGIR*. 78–88. https://doi.org/10.1145/3539618.3591689

[10] Hui Fang, Danning Zhang, Yiheng Shu, and Guibing Guo. 2020. Deep learning for sequential recommendation: Algorithms, influential factors, and evaluations. *TOIS* 1 (2020), 1–42. https://doi.org/10.1145/3426723

[11] Ruining He and Julian McAuley. 2016. Fusing similarity models with markov chains for sparse sequential recommendation. In *Proceedings of ICDM*. 191–200. https://ieeexplore.ieee.org/abstract/document/7837843

[12] Zhuangzhuang He, Yifan Wang, Yonghui Yang, Peijie Sun, Le Wu, Haoyue Bai, Jinqi Gong, Richang Hong, and Min Zhang. 2024. Double correction framework for denoising recommendation. In *Proceedings of SIGKDD*. 1062–1072.

[13] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based Recommendations with Recurrent Neural Networks. In *Proceedings of ICLR*. http://arxiv.org/abs/1511.06939

[14] Yupeng Hou, Shanlei Mu, Wayne Xin Zhao, Yaliang Li, Bolin Ding, and Ji-Rong Wen. 2022. Towards Universal Sequence Representation Learning for Recommender Systems. In *Proceedings of SIGKDD*. 585–593. https://doi.org/10.1145/3534678.3539381

[15] Jie Huang and Kevin Chen-Chuan Chang. 2023. Towards Reasoning in Large Language Models: A Survey. In *Proceedings of ACL Findings*. 1049–1065. https://aclanthology.org/2023.findings-acl.67/

[16] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *Proceedings of ICDM*. 197–206. https://ieeexplore.ieee.org/document/8594844

[17] Walid Krichene and Steffen Rendle. 2020. On Sampled Metrics for Item Recommendation. In *Proceedings of SIGKDD*. 1748–1757. https://dl.acm.org/doi/10.1145/3394486.3403226

[18] Xuan Nhat Lam, Thuc Vu, Trong Duc Le, and Anh Duc Duong. 2008. Addressing cold-start problem in recommendation systems. In *Proceedings of the 2nd International Conference on Ubiquitous Information Management and Communication*. 208–211. https://doi.org/10.1145/1352793.1352837

[19] Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Xiangyang Li, Chenxu Zhu, Huifeng Guo, Yong Yu, Ruiming Tang, et al. 2023. How Can Recommender Systems Benefit from Large Language Models: A Survey. *ArXiv preprint* abs/2306.05817 (2023). https://arxiv.org/abs/2306.05817

[20] Yujie Lin, Chenyang Wang, Zhumin Chen, Zhaochun Ren, Xin Xin, Qiang Yan, Maarten de Rijke, Xiuzhen Cheng, and Pengjie Ren. 2023. A self-correcting sequential recommender. In *Proceedings of WWW*. 1283–1293. https://doi.org/10.1145/3543507.3583479

[21] Chang Liu, Xiaoguang Li, Guohao Cai, Zhenhua Dong, Hong Zhu, and Lifeng Shang. 2021. Noninvasive self-attention for side information fusion in sequential recommendation. In *Proceedings of AAAI*. 4249–4256. https://doi.org/10.1609/aaai.v35i5.16549

[22] Qiang Liu, Shu Wu, Diyi Wang, Zhaokang Li, and Liang Wang. 2016. Context-aware sequential recommendation. In *Proceedings of ICDM*. 1053–1058. https://doi.org/10.1109/ICDM.2016.0135

[23] Zhenghao Liu, Sen Mei, Chenyan Xiong, Xiaohua Li, Shi Yu, Zhiyuan Liu, Yu Gu, and Ge Yu. 2023. Text Matching Improves Sequential Recommendation by Reducing Popularity Biases. In *Proceedings of CIKM*. 1534–1544. https://dl.acm.org/doi/abs/10.1145/3583780.3615077

[24] Zhenghao Liu, Kaitao Zhang, Chenyan Xiong, Zhiyuan Liu, and Maosong Sun. 2021. OpenMatch: An Open Source Library for Neu-IR Research. In *Proceedings of SIGIR*. 2531–2535. https://dl.acm.org/doi/abs/10.1145/3404835.3462789

[25] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton Van Den Hengel. 2015. Image-based recommendations on styles and substitutes. In *Proceedings of SIGIR*. 43–52. https://doi.org/10.1145/2766462.2767755

[26] Feiyang Pan, Shuokai Li, Xiang Ao, Pingzhong Tang, and Qing He. 2019. Warm Up Cold-start Advertisements: Improving CTR Predictions via Learning to Learn ID Embeddings. In *Proceedings of SIGIR*. 695–704. https://doi.org/10.1145/3331184.3331268

[27] Yuqi Qin, Pengfei Wang, and Chenliang Li. 2021. The world is binary: Contrastive learning for denoising next basket recommendation. In *Proceedings of SIGIR*. 859–868. https://dl.acm.org/doi/abs/10.1145/3404835.3462836

[28] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *JMLR* (2020), 140:1–140:67. https://www.jmlr.org/papers/v21/20-074.html

[29] Steffen Rendle. 2010. Factorization Machines. In *Proceedings of ICDM*. 995–1000.

[30] Andrew I. Schein, Alexandrin Popescul, Lyle H. Ungar, and David M. Pennock. 2002. Methods and metrics for cold-start recommendations. In *Proceedings of SIGIR*. 253–260. https://dl.acm.org/doi/abs/10.1145/564376.564421

[31] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In *Proceedings of CIKM*. 1441–1450. https://doi.org/10.1145/3357384.3357895

[32] Yatong Sun, Bin Wang, Zhu Sun, and Xiaochun Yang. 2021. Does Every Data Instance Matter? Enhancing Sequential Recommendation by Eliminating Unreliable Data.. In *Proceedings of IJCAI*. 1579–1585. https://www.ijcai.org/proceedings/2021/0218

[33] Jiaxi Tang and Ke Wang. 2018. Personalized Top-N Sequential Recommendation via Convolutional Sequence Embedding. In *Proceedings of WSDM*. 565–573. https://doi.org/10.1145/3159652.3159656

[34] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *ArXiv preprint* abs/2302.13971 (2023). https://arxiv.org/abs/2302.13971

[35] Shoujin Wang, Liang Hu, Yan Wang, Longbing Cao, Quan Z. Sheng, and Mehmet A. Orgun. 2019. Sequential Recommender Systems: Challenges, Progress and Prospects. In *Proceedings of IJCAI*. 6332–6338. https://doi.org/10.24963/ijcai.2019/883

[36] Wenjie Wang, Fuli Feng, Xiangnan He, Liqiang Nie, and Tat-Seng Chua. 2021. Denoising implicit feedback for recommendation. In *Proceedings of WSDM*. 373–381.

[37] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, and Jamie Brew. 2019. HuggingFace's Transformers: State-of-the-art Natural Language Processing. *ArXiv preprint* abs/1910.03771 (2019). https://arxiv.org/abs/1910.03771

[38] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-Pack: Packaged Resources To Advance General Chinese Embedding. arXiv:2309.07597 [cs.CL]

[39] Yueqi Xie, Peilin Zhou, and Sunghun Kim. 2022. Decoupled Side Information Fusion for Sequential Recommendation. In *Proceedings of SIGIR*. 1611–1621. https://dl.acm.org/doi/abs/10.1145/3477495.3531963

[40] An Yan, Shuo Cheng, Wang-Cheng Kang, Mengting Wan, and Julian J. McAuley. 2019. CosRec: 2D Convolutional Neural Networks for Sequential Recommendation. In *Proceedings of CIKM*. 2173–2176. https://doi.org/10.1145/3357384.3358113

[41] Cheng Yang, Maosong Sun, Wayne Xin Zhao, Zhiyuan Liu, and Edward Y Chang. 2017. A neural network approach to jointly modeling social networks and mobile trajectories. *TOIS* 4 (2017), 1–28. https://dl.acm.org/doi/abs/10.1145/3041658

[42] Shenghao Yang, Chenyang Wang, Yankai Liu, Kangping Xu, Weizhi Ma, Yiqun Liu, Min Zhang, Haitao Zeng, Junlan Feng, and Chao Deng. 2023. Collaborative Word-based Pre-trained Item Representation for Transferable Recommendation. In *Proceedings of ICDM*. 728–737. https://doi.org/10.1109/ICDM58522.2023.00082

[43] Yuhao Yang, Chao Huang, Lianghao Xia, Chunzhen Huang, Da Luo, and Kangyi Lin. 2023. Debiased contrastive learning for sequential recommendation. In *Proceedings of WWW*. 1063–1073. https://dl.acm.org/doi/abs/10.1145/3543507.3583361

[44] Shi Yu, Zhenghao Liu, Chenyan Xiong, and Zhiyuan Liu. 2023. OpenMatch-v2: An All-in-one Multi-Modality PLM-based Information Retrieval Toolkit. In *Proceedings of SIGIR*. 3160–3164. https://dl.acm.org/doi/abs/10.1145/3539618.3591813

[45] Fajie Yuan, Alexandros Karatzoglou, Ioannis Arapakis, Joemon M. Jose, and Xiangnan He. 2019. A Simple Convolutional Generative Network for Next Item

Recommendation. In *Proceedings of WSDM*. 582–590. https://doi.org/10.1145/3289600.3290975
[46] Jiahao Yuan, Zihan Song, Mingyou Sun, Xiaoling Wang, and Wayne Xin Zhao. 2021. Dual sparse attention network for session-based recommendation. In *Proceedings of AAAI*. 4635–4643. https://doi.org/10.1609/aaai.v35i5.16593
[47] Xu Yuan, Dongsheng Duan, Lingling Tong, Lei Shi, and Cheng Zhang. 2021. ICAI-SR: Item Categorical Attribute Integrated Sequential Recommendation. In *Proceedings of SIGIR*. 1687–1691. https://dl.acm.org/doi/abs/10.1145/3404835.3463060
[48] Zheng Yuan, Fajie Yuan, Yu Song, Youhua Li, Junchen Fu, Fei Yang, Yunzhu Pan, and Yongxin Ni. 2023. Where to Go Next for Recommender Systems? ID- vs. Modality-based Recommender Models Revisited. In *Proceedings of SIGIR*. 2639–2649. https://dl.acm.org/doi/abs/10.1145/3539618.3591932
[49] Chi Zhang, Rui Chen, Xiangyu Zhao, Qilong Han, and Li Li. 2023. Denoising and prompt-tuning for multi-behavior recommendation. In *Proceedings of WWW*. 1355–1363. https://dl.acm.org/doi/abs/10.1145/3543507.3583513
[50] Chi Zhang, Yantong Du, Xiangyu Zhao, Qilong Han, Rui Chen, and Li Li. 2022. Hierarchical item inconsistency signal learning for sequence denoising in sequential recommendation. In *Proceedings of CIKM*. 2508–2518. https://dl.acm.org/doi/abs/10.1145/3511808.3557348
[51] Chi Zhang, Qilong Han, Rui Chen, Xiangyu Zhao, Peng Tang, and Hongtao Song. 2024. SSDRec: Self-Augmented Sequence Denoising for Sequential Recommendation. In *Proceedings of ICDE*. 803–815. https://doi.org/10.1109/ICDE60146.2024.00067
[52] Kaike Zhang, Qi Cao, Yunfan Wu, Fei Sun, Huawei Shen, and Xueqi Cheng. 2025. Personalized Denoising Implicit Feedback for Robust Recommender System. In *Proceedings of WWW*. 4470–4481.
[53] Lingzi Zhang, Xin Zhou, Zhiwei Zeng, and Zhiqi Shen. 2024. Dual-view whitening on pre-trained text embeddings for sequential recommendation. In *Proceedings of AAAI*, Vol. 38. 9332–9340.
[54] Mengqi Zhang, Shu Wu, Xueli Yu, Qiang Liu, and Liang Wang. 2022. Dynamic graph neural networks for sequential recommendation. *TKDE* (2022). https://doi.org/10.1109/TKDE.2022.3151618
[55] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. Tinyllama: An open-source small language model. *ArXiv preprint* abs/2401.02385 (2024). https://arxiv.org/abs/2401.02385
[56] Yuhui Zhang, Hao Ding, Zeren Shui, Yifei Ma, James Zou, Anoop Deoras, and Hao Wang. 2021. Language Models as Recommender Systems: Evaluations and Limitations. In *Proceedings of NeurIPS Workshop*. https://openreview.net/forum?id=hFx3fY7-m9b
[57] Wayne Xin Zhao, Shanlei Mu, Yupeng Hou, Zihan Lin, Yushuo Chen, Xingyu Pan, Kaiyuan Li, Yujie Lu, Hui Wang, Changxin Tian, et al. 2021. Recbole: Towards a unified, comprehensive and efficient framework for recommendation algorithms. In *Proceedings of CIKM*. 4653–4664. https://dl.acm.org/doi/abs/10.1145/3459637.3482016
[58] Kun Zhou, Hui Wang, Wayne Xin Zhao, Yutao Zhu, Sirui Wang, Fuzheng Zhang, Zhongyuan Wang, and Ji-Rong Wen. 2020. S3-Rec: Self-Supervised Learning for Sequential Recommendation with Mutual Information Maximization. In *Proceedings of CIKM*. 1893–1902. https://doi.org/10.1145/3340531.3411954
[59] Kun Zhou, Hui Yu, Wayne Xin Zhao, and Ji-Rong Wen. 2022. Filter-enhanced MLP is all you need for sequential recommendation. In *Proceedings of WWW*. 2388–2399. https://dl.acm.org/doi/abs/10.1145/3485447.3512111

Table 6: Statistics of the Datasets Used for Modeling the Sequential Recommendation Module (M_{Rec}).

Dataset	Data Information			Split		
	#Users	#Items	#Actions	Train	Dev	Test
Beauty	22,363	12,101	198,502	131,413	22,363	22,363
Sports	35,598	18,357	296,337	189,543	35,598	35,598
Toys	19,412	11,924	167,597	109,361	19,412	19,412
Yelp	30,499	20,068	317,182	225,685	30,499	30,499

Table 7: Statistics of Training Data Used in the Filter Module (M_{Filter}). The data is used for optimizing item representations to estimate their similarities.

Dataset	Data Information			Split	
	#Users	#Items	#Actions	Train	Dev
Beauty	22,363	12,101	198,502	225,685	19,412
Sports	35,598	18,357	296,337	225,685	19,412
Toys	19,142	11,924	167,597	225,685	19,412
Yelp	30,499	20,068	317,182	225,685	19,412
Total	107,602	62,450	979,618	902,740	77,648

A Appendix

A.1 License

We show the licenses of the datasets that we use. Yelp[5] uses Apache License 2.0, while Amazon Product shows its terms of use on website[6]. All of these licenses and agreements allow their data for academic use.

A.2 Additional Experimental Details of Data Preprocessing

In our experiments, we employ Recbole [57] for preprocessing the data for sequential recommendation modeling, maintaining the same experimental settings as prior works [4, 23, 39, 58]. Specifically, each user-item interaction sequence $\{v_1, \ldots, v_t\}$ is processed using the leave-one-out strategy [5, 31, 39, 58] to construct the training, development, and testing sets. In detail, the testing set predicts the t-th item v_t based on the previously interacted items $v_{1:t-1}$ of the user, while the development set predicts the $t-1$-th item v_{t-1} given the interactions $v_{1:t-2}$. The training set predicts v_i using $v_{1:i-1}$, where $1 < i < t-1$. Items and users with fewer than five interactions are filtered out, and all user-item interactions are treated as implicit feedback [4, 5, 23, 31, 39, 58]. The data statistics are shown in Table 6.

Then, we construct the dataset for training the language model to represent items for estimating their similarities (Eq. 10). We start with the training and development sets used in sequential recommendation modeling and further process them for the Next Item Prediction (NIP) task. We first resample user-item interaction sequences from different datasets, namely Beauty, Sports, Toys, and Yelp. Then, we balance the number of data points to construct the training and development sets. This approach mitigates the risk of overfitting caused by data imbalance in different recommendation tasks.

[5] https://www.yelp.com/dataset
[6] http://jmcauley.ucsd.edu/data/amazon/

As shown in Table 7, the data statistics for both the training and development sets used in item representation learning are presented.

Following previous studies [23, 39], we conduct evaluations in the full ranking testing scenario [6, 17], where items are ranked across the entire item set rather than sampling subsets. This evaluation set mitigates the inconsistencies often observed in sampled evaluations, offering a more realistic and comprehensive assessment of recommendation performance.

A.3 Theoretical Analysis of Stable User Preference Modeling via LISRec

Preliminary. Let $\mathcal{H} = \{v_1, \ldots, v_n\}$ be the full interaction history of a user. The consistent user preference set P can be represented:

$$P \subseteq \mathcal{H}, |P| = m, \tag{15}$$

and the remaining items set can be denoted:

$$N = \mathcal{H} \setminus P, |N| = n - m. \tag{16}$$

Then, we denote the user interaction graph as an undirected graph:

$$G_{\mathcal{U}} = (\mathcal{H}, \mathcal{E}). \tag{17}$$

Here $(v_i, v_j) \in \mathcal{E} \iff \cos(\vec{v}_i, \vec{v}_j) = S_{ij} \geq \tau$, where the fixed threshold $\tau \in (0, 1)$. In the user interaction graph $G_{\mathcal{U}}$, if every pair of nodes in a set can be connected by a path, then this set of nodes is called a "connected component". We denote the collection of all connected components as:

$$\mathcal{C} = \{C_1, C_2, \ldots, C_\ell\}, \tag{18}$$

where each connected component $C_k \subseteq \mathcal{H}$ and $C_i \cap C_j = \emptyset$ for all $i \neq j$. Then, we denote

$$C_{\max} = \arg \max_{1 \leq k \leq \ell} |C_k| \tag{19}$$

as the connected component with the maximum number of nodes. Above all, our main claim is that the maximum connected component $C_{\max}$ is exactly the user preference set P.

Assumptions. We need the following three weakest conditions to support the subsequent derivation.

(i) The preference nodes are mutually reachable in $G_{\mathcal{U}}$:

$$\forall v_i, v_j \in P : v_i \sim_\tau v_j \tag{20}$$

(ii) There is no edge between any noise node in N and any preferred node in P:

$$\forall v_i \in P, v_j \in N : S_{ij} < \tau; \tag{21}$$

(iii) Even if noise nodes have high similarity and form small clusters, their scale will not exceed that of the preference set P:

$$\forall \{C_1^N, \ldots, C_q^N\} \subseteq \mathcal{C} : \max_{1 \leq r \leq q} |C_r^N| < m. \tag{22}$$

Lemmas. The following three lemmas correspond to assumptions (i)-(iii), respectively.

(1) *The preference set induces a connected component.* By assumption (i), every pair of nodes in P can reach each other through edges whose similarities exceed τ, hence all nodes of P lie in the same connected component of $G_{\mathcal{U}}$. Moreover, by assumption (ii), no node in N is adjacent to any node in P, so that component contains only the nodes of P.

(2) *Noise nodes and preference nodes are not in the same component.* By assumption (ii), $\forall v_i \in P, v_j \in N$, then $S_{ij} < \tau$, which implies that there is no direct edge between v_i and v_j. If a noise node and a preferred node are in the same connected component, there must exist a path between the two nodes. However, as long as there is one transition from a noise node to a preferred node, it will violate assumption (ii). Therefore, such a path cannot exist. Consequently, all noise nodes and the connected component belong to different components.

(3) *There is an upper bound on the scale of the noise cluster.* By assumption (iii), any connected component C_r^N composed solely of noise nodes satisfies $|C_r^N| < m$.

Proof of Proposition. We prove the two directions $v \in P \iff v \in C_{\max}$.

If $v \in P$, then $v \in C_{\max}$. By Lemma (1) and Lemma (3), any two nodes in P are connected by an edge, so that P itself forms a connected component, which we denote by C_P, and $|C_P| = |P| = m$. Then, each noise node has no edges with P, and there are no edges between noise nodes. So all noise nodes are either isolated or form small components with size less than m. Therefore, among all components, $|C_P|$ is the largest. Hence $C_{\max} = C_P$, and thus $P \subseteq C_{\max}$.

If $v \in C_{\max}$, then $v \in P$. Suppose, for contradiction, there exists $v_k \in N \cap C_{\max}$. Since $C_{\max} = C_P$, v_k is connected to some $v_i \in P$, and there must be a path passing through several edges. However, by Lemma (2) and Lemma (3), there is no noise node connected to any node in P, and there are no edges between noise nodes. So, it is impossible to construct a path from v_k to v_i. Therefore, noise nodes cannot appear in $C_{\max}$. Hence, $C_{\max} \subseteq P$.

Thus, we can prove that $C_{\max} = P$.